

# Jamii Savings Integration Platform — Solution Design Document

**Author:** Emmanuel · **Date:** 7 October 2026

## 1. Document control and design summary

| Field      | Value                                        |
|------------|----------------------------------------------|
| Version    | 1.4 (Final draft)                            |
| Author     | Emmanuel                                     |
| Implements | BRD v1.4, Jamii Savings Integration Platform |
| Status     | Final draft                                  |

Three MI integrations sit behind one APIM gateway, and one Jenkins pipeline deploys all of them to a docker-compose stack. Every API returns the same error contract and masks sensitive data in its logs.

### Key decisions

| Area             | Decision                                                                                                                                                                        |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Balance backend  | H2 2.x in server (TCP) mode in its own container, exposed through an MI Data Service with parameterised queries                                                                 |
| Customer backend | Jamii mock backends (Spring Boot 3.5, JDK 21): GET /customers/{id} with bank-shaped records and rule-driven 404, 500 and delay; built and deployed separately                   |
| Loan backend     | Jamii mock backends: credit-check SOAP service (CheckEligibility) with rule-driven client faults, server faults and delay                                                       |
| Security         | OAuth2 for Balance and Loan; API Key for Customer                                                                                                                               |
| Throttling       | Balance and Customer on Gold; Loan on a custom LoanCheck-10PerMin tier                                                                                                          |
| Gateway policy   | Custom APIM operation policy on the Balance API: correlation ID and channel header injection                                                                                    |
| Packaging        | Every component ships as a versioned Docker image (MI with CARs, APIM, H2, ELK, Jenkins agent), pushed to a local registry; the mock backends are built and deployed separately |
| Release safety   | Validate everything first, deploy, smoke test, roll back on any failure                                                                                                         |
| Edge             | nginx as the single public entry, TLS with a wildcard certificate, one subdomain per service under jamii.daqiqtech.co.ke (section 8.3); stretch goal                            |

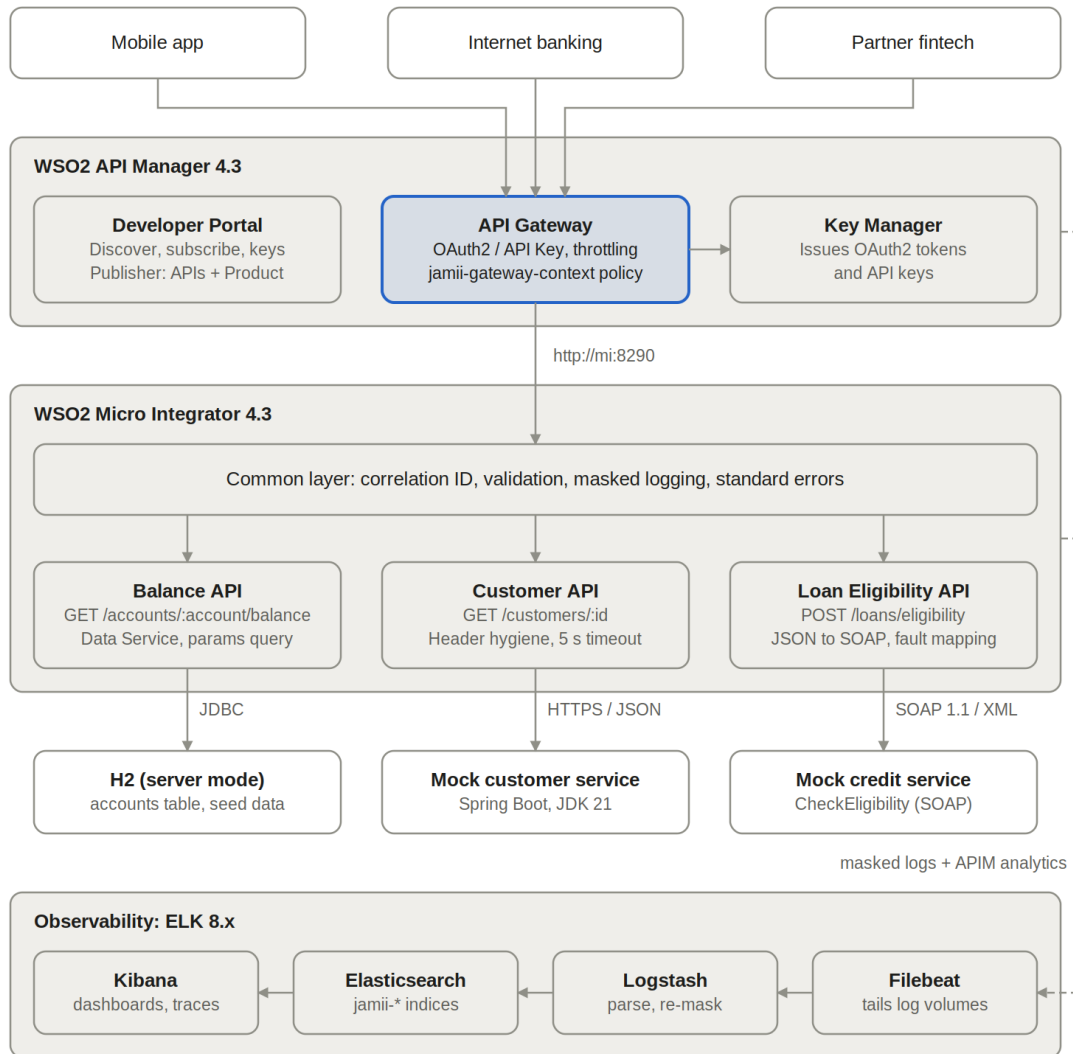
**Observability:** MI and APIM logs flow through Filebeat and Logstash into Elasticsearch. Kibana provides dashboards and end-to-end tracing by correlation ID (section 8.2). ELK and the nginx edge are stretch goals, started only after the core pipeline is green, to hold the 12 Oct submission.

**Deployment model: Option C, self-managed.** APIM and MI both run as self-managed containers, not on WSO2's managed cloud. Their images are built with docker compose build from product distributions downloaded from wso2.com, using our own Dockerfiles. The team therefore owns hosting, patching, keystores, credentials and backups (section 8.1).

## 2. Architecture overview

Consumers only ever talk to the APIM gateway, through nginx at api.jamii.daqiqtech.co.ke. MI is an internal layer that normalises three very different backends behind one contract.

Every consumer call passes nginx and the APIM gateway before it reaches MI



*Logical architecture: consumers, APIM, MI, backends and ELK*

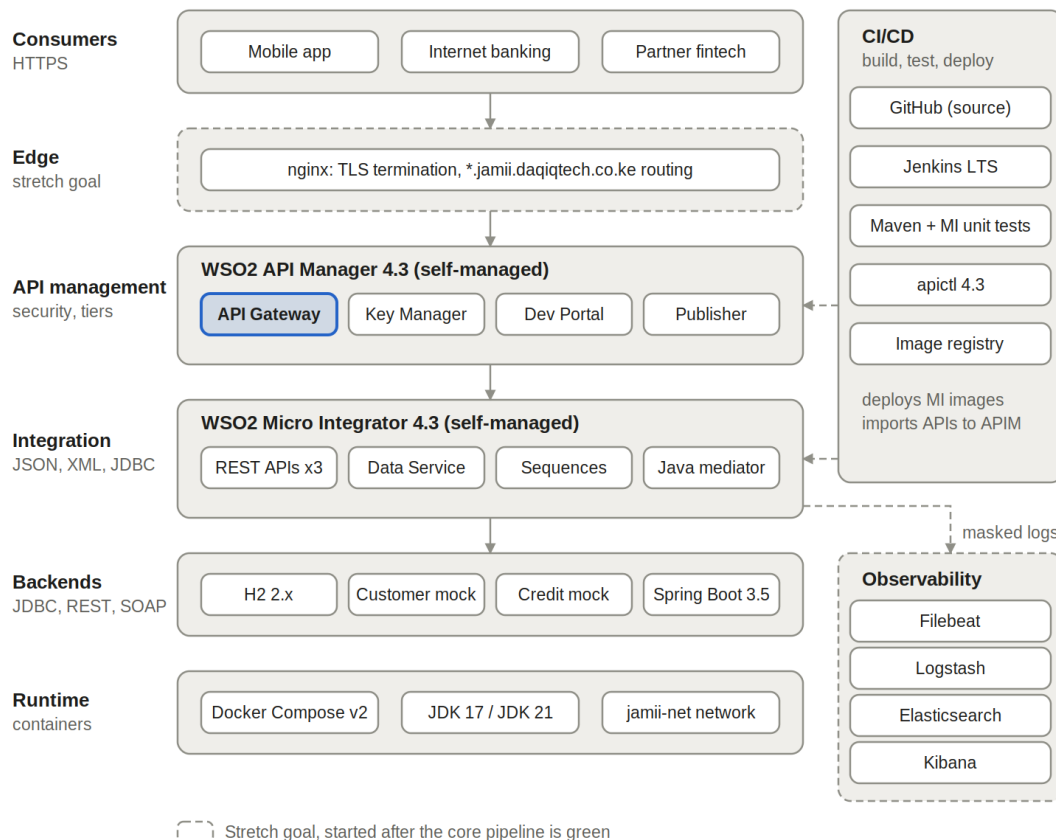
The gateway handles identity, quotas and edge policy. MI handles validation, transformation, masking and error mapping. The backends never see consumer credentials. APIM and MI both ship masked logs to the ELK stack (dashed line).

## 2.1 High-level design and technologies

The HLD shows the solution as layers, each with the technology that implements it. Requests flow down through the layers. CI/CD deploys into the two WSO2 layers from the side, and observability collects masked logs from the side.

## One gateway, one integration layer, one pipeline

Jamii Savings high-level design, with the technology used at each layer



High-level design: layers and technologies

## 3. Technology stack

These are the target versions. On Day 1, confirm the latest matching MI/APIM/apictl release and pin it in .env.

| Layer              | Tool                                                                               | Version (pin)          | Purpose                                         |
|--------------------|------------------------------------------------------------------------------------|------------------------|-------------------------------------------------|
| Integration        | WSO2 Micro Integrator                                                              | 4.3.0 (on JDK 17)      | Mediation, data services                        |
| Tooling            | WSO2 MI VS Code extension + Maven 3.9                                              | latest / 3.9.x         | Project, CAR build, unit tests                  |
| API management     | WSO2 API Manager (all-in-one)                                                      | 4.3.0 (on JDK 17)      | Publisher, Dev Portal, Gateway, Key Manager     |
| CLI                | apictl                                                                             | 4.3.x                  | Export/import APIs and products per environment |
| Database           | H2                                                                                 | 2.x, server (TCP) mode | Accounts data                                   |
| Log shipping       | Filebeat                                                                           | 8.x                    | Tails MI and APIM log volumes                   |
| Log processing     | Logstash                                                                           | 8.x                    | Parse, enrich, re-mask, route to indices        |
| Search and storage | Elasticsearch                                                                      | 8.x, single node       | Log and APIM analytics indices                  |
| Dashboards         | Kibana                                                                             | 8.x                    | Operations dashboard, trace by correlation ID   |
| Containers         | Docker, Compose v2, registry:2                                                     | latest                 | Build, tag, store and run every image           |
| CI/CD              | Jenkins LTS (custom image: JDK 21 and JDK 17, Maven, apictl, curl, jq, Docker CLI) | LTS                    | Declarative pipeline                            |
| Tests              | MI unit testing framework,                                                         | latest                 | Functional, contract, schema                    |

| Layer         | Tool                                                   | Version (pin)                                       | Purpose                                                                      |
|---------------|--------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------|
|               | gateway-check.sh (curl)                                |                                                     | tests                                                                        |
| Edge          | nginx                                                  | stable (pinned)                                     | TLS termination, host-based routing, IP allowlists                           |
| Backends      | Jamii mock backends (Spring Boot, Spring Web Services) | 3.5.x, JDK 21, Maven, jar                           | Customer REST and credit-check SOAP mocks with rule-driven faults and delays |
| Java runtimes | Eclipse Temurin                                        | 17 (WSO2 MI, APIM); 21 (H2, mock backends, Jenkins) | The WSO2 start scripts accept JDK 11 to 17 only                              |

All four Elastic components are pinned to the same minor version.

## 4. Component design per API

Each API is an MI REST API artifact that calls shared sequences for validation, logging and errors. Only the backend call and the transformation differ.

### 4.1 Balance API (/accounts/{accountNumber}/balance)

| Artifact       | Name                | Responsibility                                                                                                    |
|----------------|---------------------|-------------------------------------------------------------------------------------------------------------------|
| Datasource     | JamiiAccountsDS     | JDBC pool to H2 at jdbc:h2:tcp://h2:9092/jamii (H2 driver jar baked into the MI image); credentials from env vars |
| Data Service   | AccountsDataService | Query getAccountByNumber: SELECT account_number, status, balance, currency FROM accounts WHERE account_number = ? |
| REST API       | BalanceAPI          | Validates path, calls data service, builds envelope                                                               |
| Fault sequence | Common_FaultSeq     | Maps SQL/connection errors to 503 DB_UNAVAILABLE                                                                  |

Flow: validate `^[0-9]{10,16}$` (else 400), call the data service. Zero rows returns 404 ACCOUNT\_NOT\_FOUND. One row is transformed with a PayloadFactory into the BRD envelope, with `requestId` = correlation ID and `timestamp` = UTC now. A JDBC or connection error triggers the fault sequence and returns 503.

### 4.2 Customer API (/customers/{customerId})

| Artifact      | Name                 | Responsibility                                                                                                                     |
|---------------|----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| HTTP endpoint | CustomerBackendEP    | http://jamii-mock-backends:8080/customers/{uri.var.customerId} (base URL from CUSTOMER_BACKEND_URL); timeout 5000 ms, action fault |
| REST API      | CustomerAPI          | Header manipulation, pass-through body, 404 mapping                                                                                |
| Sequence      | Common_HeaderHygiene | Add/propagate X-Correlation-ID; strip X-Internal-*, Server, X-Powered-By, Set-Cookie                                               |

Flow: validate `^[0-9]{1,10}$` (else 400), apply header hygiene and call the endpoint. A 200 body is passed through and the X-Correlation-ID response header is set. A backend 404 returns 404 CUSTOMER\_NOT\_FOUND. A timeout returns 504 BACKEND\_TIMEOUT. Connection refused or DNS failure triggers the fault sequence and returns 503 BACKEND\_UNAVAILABLE. The backend URL is a config property, so it can point at any deployed instance of the mock backends without a code change.

**Mock customer rules.** The Jamii mock backends service returns records shaped like a Kenyan bank's customer file, so masking is exercised on national ID and phone, not just names. Every response also carries Server, X-Powered-By and X-Internal-\* headers, which MI must strip.

| Request                                 | Mock response                                                                     | Purpose                                         |
|-----------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------|
| GET /customers/1001 to 1005             | 200 with customerId, fullName, nationalId, phone (+254), email, kycStatus, branch | Happy path and PII masking                      |
| GET /customers/9999 (or any unknown ID) | 404                                                                               | CUSTOMER_NOT_FOUND mapping                      |
| GET /customers/5000                     | 500                                                                               | 502 BAD_GATEWAY mapping                         |
| GET /customers/5004                     | 200 after a 7 s delay                                                             | Exceeds the 5 s timeout, so 504 BACKEND_TIMEOUT |

The mock backends are a separate Spring Boot project with its own Dockerfile and docker-compose file, built and deployed independently of the integration stack (section 8.4).

### 4.3 Loan Eligibility API (POST /loans/eligibility)

Request:

```
{ "customerId": "7", "monthlyIncome": 120000, "existingMonthlyDebt": 15000,
  "requestedMonthlyInstalment": 25000 }
```

Mapping to the credit-check SOAP operation CheckEligibility (namespace <http://jamii.daqiqtech.co.ke/credit/v1>, WSDL at /ws/creditCheck.wsdl):

- The four JSON fields map one-to-one to the SOAP request elements, built with a PayloadFactory.
- The credit engine computes debt-to-income % = (existing debt + instalment) × 100 ÷ income, and approves at 40% or less.
- MI does not make the decision. It mediates the protocol, maps decision, debtToIncomePercent, maxAllowedPercent and referenceId back to JSON, derives eligible, and maps faults.

Response:

```
{ "customerId": "7", "decision": "ELIGIBLE", "eligible": true, "debtToIncomePercent": 33,
  "maxAllowedPercent": 40, "creditReference": "CRD-7F3A9C21", "requestId": "...", "timestamp":
  "..." }
```

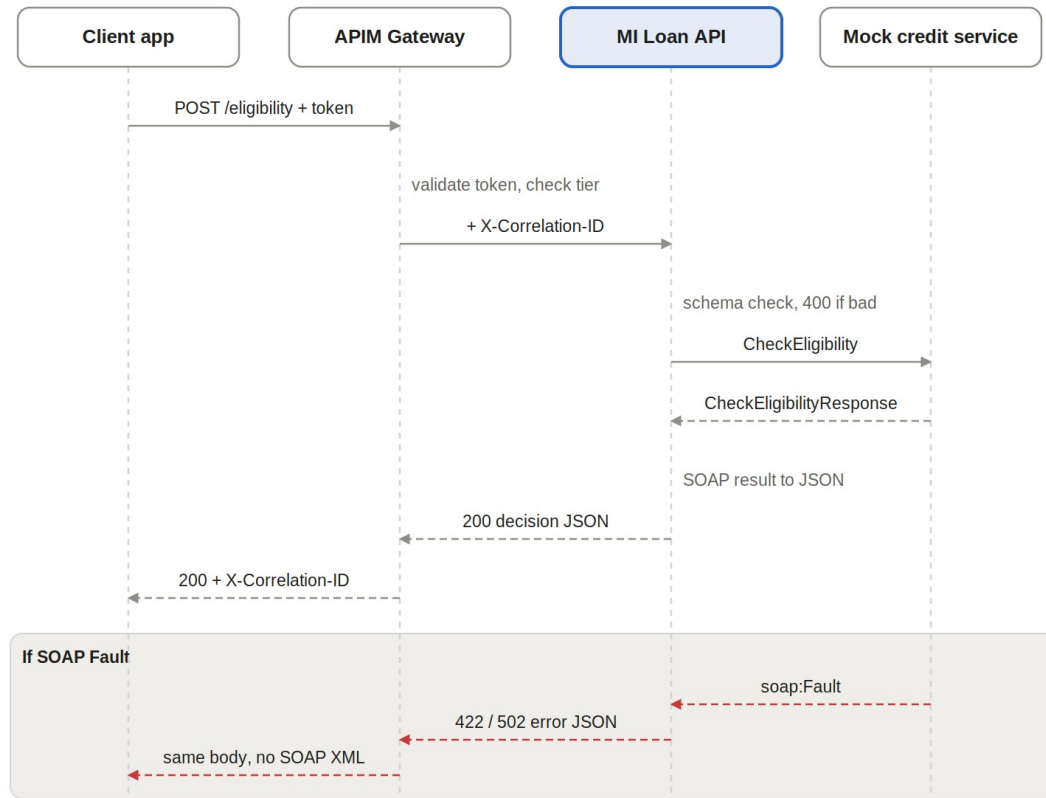
Why a purpose-built SOAP mock: a credit-check operation gives the SOAP call real lending meaning, and the mock produces both kinds of SOAP fault, plus a slow answer, on demand. The trade-off is that it is a service we built rather than an existing public one, so the README states this and justifies it.

| SOAP outcome                     | Trigger                      | REST response                                                        |
|----------------------------------|------------------------------|----------------------------------------------------------------------|
| CheckEligibilityResponse         | Normal values                | 200 with decision                                                    |
| soap:Client INVALID_INCOME       | monthlyIncome 0              | 422 ELIGIBILITY_REJECTED, "Monthly income must be greater than zero" |
| soap:Client CUSTOMER_BLACKLISTED | customerId 6000              | 422 ELIGIBILITY_REJECTED, "Applicant is not eligible for credit"     |
| soap:Server                      | customerId 5000              | 502 BAD_GATEWAY (fault string logged, never returned)                |
| No answer within 10 s            | customerId 5004 (12 s delay) | 504 BACKEND_TIMEOUT                                                  |
| Unreachable                      | Mock backends down           | 503 BACKEND_UNAVAILABLE                                              |

## 5. Loan Eligibility request flow

This is the most complex flow, because it crosses protocols twice: JSON to SOAP, then SOAP back to JSON.

### A SOAP fault never reaches the client as XML



### Loan Eligibility sequence: happy path and SOAP fault path

Solid arrows are requests and dashed arrows are responses. Red marks the fault path, where MI's fault sequence turns the SOAP fault into the standard error JSON before anything is returned.

## 6. Cross-cutting design

Shared logic lives in four reusable sequences and one template, so each API stays thin and every API behaves the same way.

| Artifact             | Type              | What it does                                                                                                                    |
|----------------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Common_InSeq         | Sequence          | Reads or generates X-Correlation-ID (UUID v4), stores it as property correlationId, records start time                          |
| Common_MaskedLog     | Sequence template | Logs a stage (REQUEST_IN, BACKEND_CALL, RESPONSE_OUT) with masked fields; params: stage, api, extra                             |
| Common_ErrorResponse | Sequence template | Builds the standard error JSON from params (httpStatus, code, message, details), sets status and correlation header, responds   |
| Common_FaultSeq      | Fault sequence    | Catches unhandled errors, classifies them by ERROR_CODE (timeout 101504, connection 101503, SQL) and calls Common_ErrorResponse |
| Common_HeaderHygiene | Sequence          | Strips internal and server-identifying headers inbound and outbound                                                             |

**Validation:** each API validates input before any backend call. Path parameters use regex. The loan body uses a JSON Schema (loan-request.schema.json, stored as a registry resource) via the Validate mediator. A failure returns 400 VALIDATION\_ERROR with field-level details. Malformed JSON is caught in the fault sequence and also returns 400, never 500.

**Masking rules:** masking happens in two layers, with no Java code. First, Common\_MaskedLog only logs whitelisted values that the API has already masked (the account number reduced to its last 4 digits, the customer ID, status codes); bodies and amounts are never logged. Second, every MI log pattern passes the message through a log4j2 %replace rule that masks any run of 10 or more digits to \*\*\*\*\* plus the last 4. The second layer also covers MI's own components, such as data-service fault messages and transport errors, which log request URLs and parameters.

| Field                                 | Rule              | Example   |
|---------------------------------------|-------------------|-----------|
| accountNumber                         | keep last 4       | *****7890 |
| name, username, email, phone, address | full mask         | ***       |
| monthlyIncome, debts, instalment      | omitted from logs | n/a       |

**Logging format:** one line per stage, key=value, for example:

```
stage=REQUEST_IN api=BalanceAPI correlationId=5b1f... account=*****7890 method=GET
stage=BACKEND_CALL api=BalanceAPI correlationId=5b1f... backend=AccountsDataService
stage=RESPONSE_OUT api=BalanceAPI correlationId=5b1f... httpStatus=200 latencyMs=42
```

## 7. API Manager design

All three APIs are published from apictl projects in Git, never hand-edited in the Publisher. Git is the source of truth.

| API                    | Context / version   | Security | Subscription tier           | Backend (MI)             |
|------------------------|---------------------|----------|-----------------------------|--------------------------|
| Jamii Account Balance  | /jamii/accounts/v1  | OAuth2   | Gold                        | http://mi:8290/accounts  |
| Jamii Customer Profile | /jamii/customers/v1 | API Key  | Gold                        | http://mi:8290/customers |
| Jamii Loan Eligibility | /jamii/loans/v1     | OAuth2   | LoanCheck-10PerMin (custom) | http://mi:8290/loans     |

**Tier reason (for README):** Loan eligibility gets a tighter custom tier because each call fans out to a slow, external SOAP dependency with no SLA, and the tier protects it from overload.

**Security reason:** OAuth2 suits user-context and financial data (Balance, Loan). API Key shows the simpler server-to-server option for partner lookups (Customer). Using both shows the trade-off.

**Custom mediation policy:** jamii-gateway-context is an APIM 4.x operation policy (.j2 + .yaml) applied on the request flow of the Balance API. It injects X-Correlation-ID if absent, so tracing starts at the edge. It adds X-Channel from the calling application's name and removes the Authorization header before forwarding, so tokens never reach MI.

**OpenAPI:** hand-written OAS 3.0 per API, including path parameter patterns, request schema, the success schema, and every error response (400, 401, 404, 422, 429, 502, 503, 504) referencing one shared ErrorResponse component.

**API Product:** Jamii Retail Banking bundles all three, with OAuth2 and the Gold tier. One subscription gives a channel app (for example the mobile app) access to all three. Partners can still subscribe to individual APIs.

### Developer Portal onboarding (consumer steps)

1. Sign in at <https://portal.dev.jamii.daqiqtech.co.ke/devportal> and browse or search the jamii tag.
2. Read the API docs and try the OpenAPI console.
3. Create an application (for example jamii-mobile) and choose a business plan.
4. Subscribe the application to the Jamii Retail Banking product, or to individual APIs.
5. Under Production Keys, generate client credentials (OAuth2) or an API key.
6. Call the gateway at <https://api.dev.jamii.daqiqtech.co.ke/jamii/...> with Authorization: Bearer <token> or the apikey header.

Until the nginx stretch is delivered, the same steps run locally against <https://localhost:9443/devportal> and the gateway at <https://localhost:8243>.

## 8. Deployment architecture

One docker compose up -d starts the whole dev environment on a single bridge network, jamii-net. Only APIM's gateway and portal ports are meant for consumers; MI is reachable directly only for debugging.

| Service               | Image                                         | Built from                                | Baked into the image                                                                                                                                  | Host ports                     | Compose profile |
|-----------------------|-----------------------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|-----------------|
| h2                    | jamii/h2:\${TAG}                              | eclipse-temurin:21-jre + H2 2.2.224 jar   | schema.sql + seed.sql, run by RunScript before the TCP server starts; database recreated on every start (H2_RESET_ON_START)                           | 9092, 8082 (console, dev only) | core            |
| mi                    | jamii/mi:\${TAG}                              | eclipse-temurin:17-jre + wso2mi-4.3.0.zip | Integration artifacts (hot-deployed for local runs; a CAR in the pipeline), H2 2.2.224 driver, 1 s timeout handler, log-masking layout (configure.sh) | 8290, 8253, 9164               | core            |
| apim                  | jamii/apim:\${TAG}                            | eclipse-temurin:17-jre + wso2am-4.3.0.zip | deployment.toml (ELK analytics on), custom policy, keystores                                                                                          | 9443, 8243, 8280               | core            |
| filebeat              | jamii/filebeat:\${TAG}                        | elastic/filebeat:8.x                      | filebeat.yml (MI and APIM inputs)                                                                                                                     | none                           | observability   |
| logstash              | jamii/logstash:\${TAG}                        | logstash:8.x                              | Pipeline: kv/json parse, re-mask, index routing                                                                                                       | 5044                           | observability   |
| elasticsearch         | stock elasticsearch:8.x                       | n/a                                       | Config via env, 1 GB heap                                                                                                                             | 9200                           | observability   |
| kibana + kibana-setup | stock kibana:8.x + jamii/kibana-setup:\${TAG} | curl image                                | dashboards.ndjson imported once at start                                                                                                              | 5601                           | observability   |
| jenkins               | jamii/jenkins:\${TAG}                         | jenkins/jenkins:lts-jdk21                 | Maven, apictl, curl, jq, Docker CLI, plugins list                                                                                                     | 8080                           | ci              |
| registry              | stock registry:2                              | n/a                                       | n/a                                                                                                                                                   | 5000                           | ci              |
| nginx                 | jamii/nginx:\${TAG}                           | nginx:stable-alpine                       | One server block per subdomain, TLS settings, IP allowlists (certificates mounted, not baked)                                                         | 80, 443                        | core            |

Images carry platform configuration; API definitions do not. The three APIs and the product are imported into APIM by apictl at deploy time. That way the same jamii/apim image is reused in every environment, and only api\_params.yaml changes.

The APIM and MI Dockerfiles unpack the downloaded zips from dist/, which is gitignored because the files are large. Each zip's SHA-256 is checked against dist/checksums.txt before the build. The containers run as a non-root wso2 user. Jenkins reads the same zips from a mounted dist volume, so CI builds exactly what runs locally.

docker compose --profile core up -d is enough for the API demo. Add --profile observability for ELK, and --profile ci for Jenkins and the registry.

## 8.1 Self-managed responsibilities (Option C)

Because both APIM and MI are self-managed, every operational concern below is ours. The table shows how this project handles each one, and what would change for production.

| Responsibility          | This project (dev)                                             | Production                                                   |
|-------------------------|----------------------------------------------------------------|--------------------------------------------------------------|
| Hosting and scaling     | docker-compose, one node per component                         | Kubernetes with WSO2 Helm charts; 2+ gateway and MI replicas |
| Product updates         | Downloaded GA distributions, version pinned, checksum verified | WSO2 Updates (subscription) applied by rebuilding images     |
| APIM internal databases | Built-in H2 (APIM default)                                     | External RDBMS for apim_db and shared_db                     |
| Keystores and TLS       | Default keystores replaced with project self-signed certs      | CA-signed certificates, secure vault for secrets             |
| Admin credentials       | admin/admin changed via env vars and Jenkins credentials       | IdP-backed users, least-privilege roles                      |
| Backups                 | Not needed (disposable stack)                                  | Scheduled DB and config backups, tested restore              |
| Monitoring              | ELK dashboards                                                 | ELK plus alerting and on-call                                |

**Environments:** each environment is a set of config values, not separate code.

| Setting                      | dev                                                    | prod (conceptual)          |
|------------------------------|--------------------------------------------------------|----------------------------|
| apictl env name              | dev                                                    | prod                       |
| APIM endpoints               | apim.dev.jamii.daqiqtech.co.ke                         | apim.jamii.daqiqtech.co.ke |
| MI backend URL in API params | http://mi:8290                                         | prod MI load balancer      |
| Backends                     | Jamii mock backends (Spring Boot, deployed separately) | real core/credit systems   |
| Deployment trigger           | automatic on main                                      | manual approval            |

API endpoints differ per environment through apictl api\_params.yaml, so the same API archive is promoted unchanged.

## 8.2 Observability (ELK), stretch

Logs are masked at the source in MI, shipped by Filebeat, parsed and re-checked by Logstash, stored in Elasticsearch, and explored in Kibana. ELK never receives an unmasked account number.

1. MI writes its masked key=value mediation logs and access logs to a shared volume. APIM, with analytics type elk in deployment.toml, writes gateway events to apim\_metrics.log.
2. Filebeat tails both volumes and tags each event with its source (mi or apim).
3. Logstash parses MI lines with the kv filter and APIM events as JSON. It sets @timestamp, re-masks anything matching \b\d{10,16}\b to its last 4 digits, and routes events to jamii-mi-\* or jamii-apim-analytics-.\*.
4. Elasticsearch runs as a single node with a 1 GB heap; an ILM policy deletes indices after 7 days.
5. kibana-setup imports data views and the dashboard on first start.

| Dashboard panel                        | Source         | Question it answers               |
|----------------------------------------|----------------|-----------------------------------|
| Requests per API per minute            | APIM analytics | Who is calling what, and how much |
| Error rate by error code               | MI logs        | What is failing, and why          |
| p95 latency per API and backend        | MI latencyMs   | Which backend is slow             |
| Throttled (429) calls by application   | APIM analytics | Is a tier too tight               |
| Trace by correlation ID (saved search) | MI + APIM      | What happened to this one request |

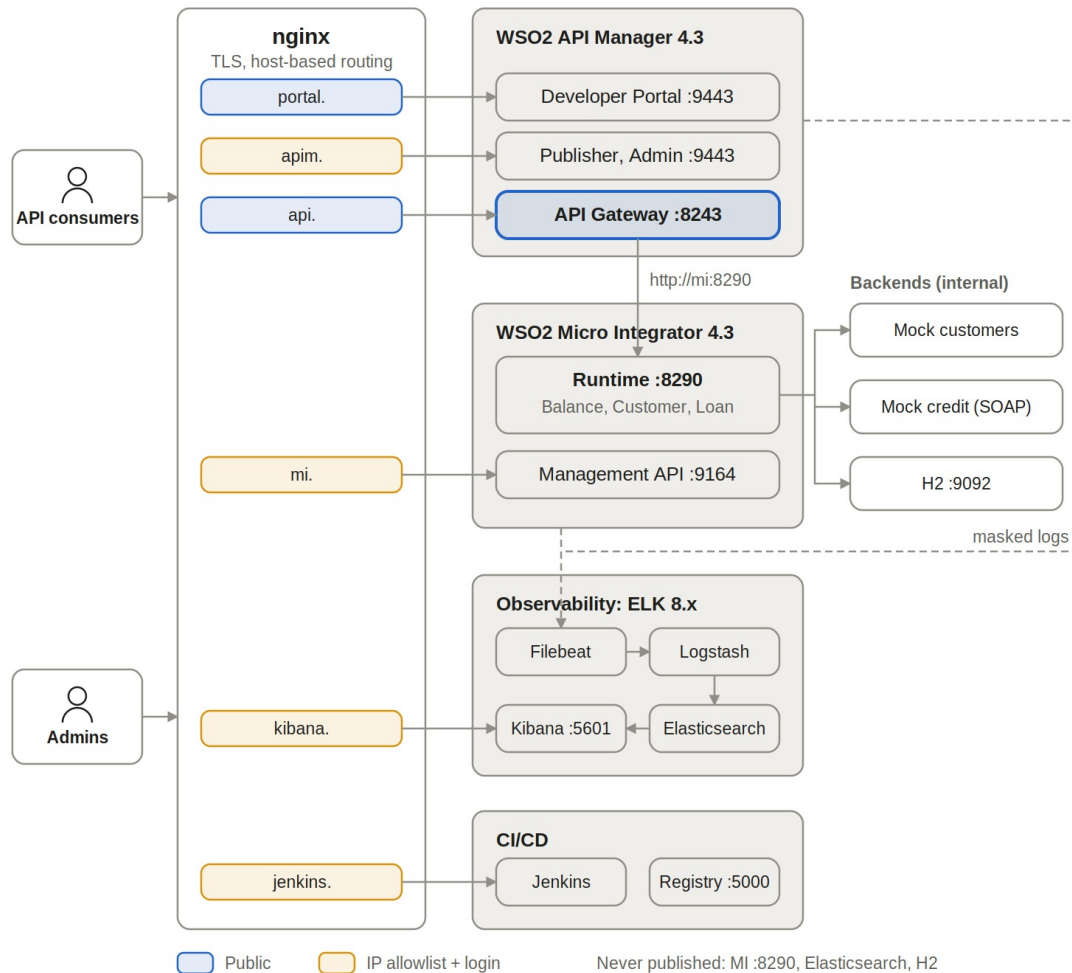
**Resource budget:** APIM 2 GB, Elasticsearch 1.5 GB, Logstash 1 GB (512 MB heap), MI 1 GB, Kibana 0.7 GB, Jenkins 1 GB, others under 0.5 GB. That is about 8 GB, so a 16 GB machine is the minimum. If memory is tight, Logstash can be dropped and Filebeat can write to an Elasticsearch ingest pipeline instead.

### 8.3 Edge topology: nginx and subdomains, stretch

nginx is the single public entry point. It terminates TLS with one wildcard certificate and routes by hostname, so no WSO2, Elastic or Jenkins port is ever exposed directly.

**nginx is the only public entry, and each service gets its own subdomain**

One wildcard TLS certificate: \*.jamii.daqiqtech.co.ke (dev: \*.dev.jamii.daqiqtech.co.ke)



*Edge topology: nginx, subdomains and internal services*

| Subdomain (prod)              | Dev equivalent                    | Routes to                                | Exposure                                             |
|-------------------------------|-----------------------------------|------------------------------------------|------------------------------------------------------|
| api.jamii.daqiqtech.co.ke     | api.dev.jamii.daqiqtech.co.ke     | apim:8243 (gateway, incl. /oauth2/token) | Public                                               |
| portal.jamii.daqiqtech.co.ke  | portal.dev.jamii.daqiqtech.co.ke  | apim:9443 /devportal                     | Public                                               |
| apim.jamii.daqiqtech.co.ke    | apim.dev.jamii.daqiqtech.co.ke    | apim:9443 /publisher, /admin, /carbon    | IP allowlist + login (Jenkins IP allowed for apictl) |
| mi.jamii.daqiqtech.co.ke      | mi.dev.jamii.daqiqtech.co.ke      | mi:9164 management API only              | IP allowlist + login                                 |
| kibana.jamii.daqiqtech.co.ke  | kibana.dev.jamii.daqiqtech.co.ke  | kibana:5601                              | IP allowlist + Elastic login                         |
| jenkins.jamii.daqiqtech.co.ke | jenkins.dev.jamii.daqiqtech.co.ke | jenkins:8080                             | IP allowlist + Jenkins login; webhook path open      |

MI's runtime (8290), Elasticsearch, H2 and the registry stay on the internal Docker network. Publishing MI's runtime would let callers bypass the gateway's security and throttling.

#### nginx and WSO2 configuration notes

- APIM must know its public hostnames. Set the server hostname, proxy port 443, and the gateway and Developer Portal URLs in deployment.toml; otherwise logins and redirects point at internal ports. Verify the exact keys against the 4.3 docs.
- WSO2 listens on HTTPS internally with self-signed certificates, so nginx uses proxy\_pass https://apim:9443 and proxy\_ssl\_verify off. This is acceptable on the internal network for dev; prod uses trusted certificates.
- Raise client\_max\_body\_size (for example to 50m) on apim. so apictl API imports are not rejected.
- Set Kibana server.publicBaseUrl and the Jenkins URL to their public hostnames.
- Locally, there is no real DNS: use mkcert for the wildcard certificate and /etc/hosts entries pointing the six hostnames to 127.0.0.1.

## 8.4 Mock backends (separate deployment)

The customer system and the credit-check engine are mocked by one Spring Boot service, jamii-mock-backends. It lives in its own repository and is built and deployed independently of the integration stack.

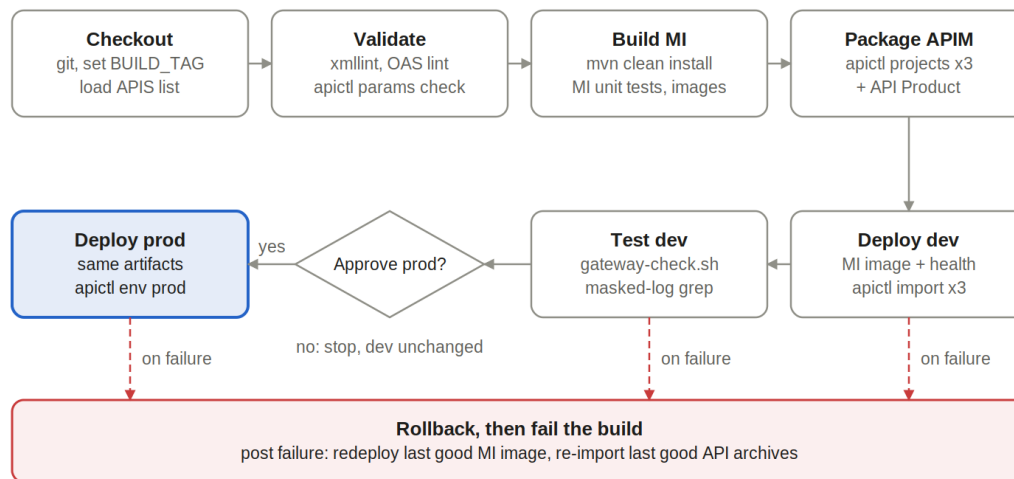
| Item      | Value                                                                                                                                                                                                                                                                                                   |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stack     | Java 21, Spring Boot 3.5 (Spring Web and Spring Web Services), Maven, executable jar                                                                                                                                                                                                                    |
| Endpoints | GET /customers/{id}, POST /ws (SOAP 1.1), GET /ws/creditCheck.wsdl, GET /actuator/health                                                                                                                                                                                                                |
| API docs  | Swagger UI at /swagger-ui.html and OpenAPI 3 spec at /v3/api-docs (springdoc) for the customer REST API; the SOAP contract is the WSDL, linked from the Swagger UI description                                                                                                                          |
| Image     | jamii/mock-backends:\${TAG}; multi-stage build (maven:3.9-eclipse-temurin-21, then eclipse-temurin:21-jre), non-root user                                                                                                                                                                               |
| Run       | Its own docker-compose.yml; host port 8081 to container 8080                                                                                                                                                                                                                                            |
| Network   | Creates the jamii-shared network; the integration stack joins it as an external network, so MI calls http://jamii-mock-backends:8080                                                                                                                                                                    |
| Elsewhere | Set CUSTOMER_BACKEND_URL and CREDIT_BACKEND_URL on MI to the deployed host                                                                                                                                                                                                                              |
| Logging   | One line per stage, tagged with the caller's X-Correlation-ID: REQUEST_IN (method, path, client IP, headers), REQUEST_BODY, RESPONSE_OUT (status, duration, headers, body; WARN for 4xx, ERROR for 5xx), plus business lines (LOOKUP, DECISION, RULE, REJECT). Credentials, PII and amounts are masked. |

Start order: the mock backends first, then docker compose --profile core up -d for the integration stack.

## 9. CI/CD pipeline design

One declarative Jenkinsfile builds and deploys all three APIs together. They are released as one unit or not at all. Every image is tagged with the build number and pushed to the local registry, so dev and prod run identical images.

Any failed stage rolls back, and prod deploys only after approval



Jenkins pipeline: stages, approval gate and rollback path

### How partial deploys are prevented

1. Everything is validated and packaged before anything is deployed, so a broken spec fails the run while all environments are untouched.
2. Before deploying, the pipeline records the current MI image tag and exports the current API archives (apictl export api) as rollback artifacts.
3. Deploy dev runs the MI image swap, waits for the MI health check, then imports the three APIs in a loop with set -e. The first failure stops the loop.
4. post { failure { sh 'scripts/rollback.sh' } } restores the previous image and re-imports the saved archives, then the build is marked FAILED.
5. Prod uses the same artifacts (no rebuild), with apictl env prod and an input approval step. In this assignment it is wired as a dry run.

Credentials (APIM admin, DB password) come from the Jenkins credentials store via withCredentials and are never echoed.

## 10. Repository structure

```

jamii-savings-integration/
├── README.md
├── Jenkinsfile
├── docker-compose.yml      # profiles: core, observability, ci
├── docker-compose.debug.yml # publishes MI 8290 on localhost for testing
├── .env.example            # versions, JDKs, passwords (no real secrets)
├── dist/                  # downloaded WSO2 zips + checksums.txt (zips gitignored)
├── db/
│   ├── schema.sql
│   └── seed.sql
├── mi/                    # MI integration project (MI 4.x layout)
│   └── src/main/wso2mi/artifacts/
│       ├── apis/          # BalanceAPI, CustomerAPI, LoanEligibilityAPI
│       ├── sequences/     # Common_InSeq, Common_HeaderHygiene, Common_FaultSeq
│       ├── templates/     # Common_ErrorResponse, Common_MaskedLog
│       ├── endpoints/     # AccountsDataServiceEP, CustomerBackendEP, CreditBackendEP
│       └── data-services/  # AccountsDataService.dbs
├── apim/
│   └── apis/
│       └── JamiiAccountBalance-v1/ # apictl project (api.yaml, OAS, policies)

```

```

├── JamiiCustomerProfile-v1/
├── JamiiLoanEligibility-v1/
├── products/JamiiRetailBanking/
├── policies/jamii-gateway-context/
├── throttling/LoanCheck-10PerMin.json
├── params/           # api_params.yaml per environment
├── docker/           # one Dockerfile + config per image
├── h2/               # entrypoint: schema + seed, TCP server
├── mi/               # Dockerfile, entrypoint.sh, configure.sh
├── apim/             # deployment.toml (ELK analytics)
├── filebeat/
├── logstash/         # pipeline/jamii.conf
├── nginx/            # conf.d per subdomain; certs gitignored
├── kibana-setup/     # dashboards.ndjson
├── jenkins/          # plugins.txt, tools
├── tests/
├── gateway/          # gateway-check.sh settings per environment
├── schemas/error-response.schema.json
├── scripts/           # mi-smoke.sh, gateway-check.sh, build-images.sh, deploy-*.sh,
rollback.sh
├── docs/             # BRD, design doc, diagrams, Postman, Jira plan

```

The Jenkinsfile loops over a single list, APIS = ['balance','customer','loan']. Adding a fourth API means adding a folder and one list entry, with no new pipeline code.

## 11. Testing strategy

Tests run in the pipeline in order of cost: fast checks first, full gateway tests last.

| Level                | Tool                                                                                                                               | What it proves                                                                                                                | Pipeline stage            |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Static               | xmllint, OAS lint (spectral), apictl validation                                                                                    | Artifacts and specs are well formed                                                                                           | Validate                  |
| Unit                 | JUnit in the mock backends (17 tests, including log masking)                                                                       | Masking rules work in isolation                                                                                               | Build                     |
| API behaviour        | MI unit testing framework (test suites in each MI module, mock services for the REST and SOAP endpoints, H2 container for Balance) | Each API: happy path, 400, 404, 422/502 SOAP fault mapping, backend down                                                      | Build (mvn clean install) |
| Contract             | JSON-path assertions in every MI error test (\$error.code, \$error.httpStatus, \$requestId, \$timestamp)                           | Every error body has the same shape                                                                                           | Build                     |
| Gateway (end to end) | scripts/gateway-check.sh (curl, exit codes) via APIM                                                                               | 401 without token, 200 with token, 429 over tier, product subscription works                                                  | Test (dev)                |
| MI smoke             | scripts/mi-smoke.sh (19 checks, MI direct)                                                                                         | Every success and error path, both timeouts, correlation ID, no leaked headers, SOAP XML or stack traces, standard error body | Test (dev)                |
| Security/logging     | grep on MI logs                                                                                                                    | No full account number or PII leaked, in log files or in Elasticsearch                                                        | Test (dev)                |

The MI unit tests need an MI runtime in CI. The Jenkins agent unpacks the MI zip from the mounted dist/ volume and points the unit test plugin at it. If the MI version cannot assert on HTTP status codes directly, tests assert on \$.error.httpStatus in the body.

Timeout scenarios, which MI mock services do not simulate well, use the mock backends' slow rules: customer 5004 waits 7 s against a 5 s timeout, and credit customer 5004 waits 12 s against 10 s. docker stop h2 covers the database. The 503 and 504 paths are tested on purpose, not by luck.

## 12. Design decisions, trade-offs and bonus design

| ADR | Decision                                                                                       | Alternative rejected                                           | Trade-off accepted                                                                                                                                                                           |
|-----|------------------------------------------------------------------------------------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01  | H2 in server mode, in its own container                                                        | H2 embedded inside MI; MySQL                                   | Not production-grade, but light; a separate container still lets the demo stop the DB to show 503                                                                                            |
| 02  | Spring Boot mock backends (customer REST and credit-check SOAP), built and deployed separately | Public mocks (JSONPlaceholder, Beeceptor, DNE Calculator)      | One more service to build and run, and not an existing public service, so the README justifies it; in return, full control of data, faults and delays, with no quotas or third-party outages |
| 03  | Credit-check SOAP contract (CheckEligibility) designed for lending                             | DNE Calculator Divide                                          | A contract we own must be documented (XSD and WSDL published); in return the SOAP call carries real lending semantics                                                                        |
| 04  | Mock backends run locally in CI on the shared Docker network                                   | Rely on public services                                        | More setup, but the demo and CI cannot be broken by a third party                                                                                                                            |
| 05  | Shared sequences/templates; two-layer masking (whitelisted logging plus a log-layout rule)     | Java class mediator; inline logic per API                      | Rule-based rather than field-aware, so short IDs rely on never being logged; in return no WSO2 Java dependencies, and MI's own internal logs are covered                                     |
| 06  | Correlation ID at both APIM (policy) and MI                                                    | Only one layer                                                 | Small duplication; MI stays safe when called directly                                                                                                                                        |
| 07  | Every component as a versioned image in a local registry                                       | Official WSO2 images with mounted config and hot-copied CARs   | Longer builds; immutable releases, and rollback becomes "run the previous tag"                                                                                                               |
| 08  | APIM config as apictl projects in Git, imported at deploy time                                 | APIs baked into the APIM image                                 | One more deploy step; one APIM image serves every environment                                                                                                                                |
| 09  | One API Product plus individual APIs                                                           | Product only                                                   | Two subscription paths to explain; fits channels and partners                                                                                                                                |
| 10  | Validate everything before deploying anything, then roll back on failure                       | Deploy each API as it builds                                   | Longer pipeline; never a partial release                                                                                                                                                     |
| 11  | ELK with Logstash, masking at source plus re-masking in Logstash                               | No central logging; Filebeat straight to Elasticsearch         | About 3 GB more RAM; searchable traces and a second masking safeguard                                                                                                                        |
| 12  | Elastic security on (password from .env), TLS off locally                                      | Security disabled                                              | Slightly more setup; mirrors how a bank would run it, while TLS is noted as a prod item                                                                                                      |
| 13  | Self-managed WSO2 (Option C), images built from downloaded distributions                       | WSO2 managed cloud; official Docker images                     | We own patching and operations; full control of versions, config and image contents                                                                                                          |
| 14  | nginx edge with one subdomain per service under jamii.daqiqtech.co.ke                          | Publish service ports directly; path-based routing on one host | One more container and certificate setup; a single TLS point, IP allowlists, no internal port exposed                                                                                        |
| 15  | WSO2 MI and APIM 4.3.0 on JDK 17                                                               | 4.4.0 on JDK 21                                                | Not the newest release; the WSO2 start scripts accept JDK 11 to 17 only, and MI and APIM stay on the same release line                                                                       |

| ADR | Decision                                                                 | Alternative rejected  | Trade-off accepted                                                                                                                |
|-----|--------------------------------------------------------------------------|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 16  | Artifacts hot-deployed into MI for local runs; CAR built in the pipeline | CAR only from day one | Two packaging paths to keep in step; MI ran end to end a day earlier                                                              |
| 17  | One MI integration project (MI 4.x layout)                               | Five Maven modules    | Less modular; matches the MI tooling layout, so it opens directly in the VS Code extension                                        |
| 18  | Synapse timeout handler runs every 1 s                                   | Default 15 s          | Slightly more background work; 5 s and 10 s timeouts are enforced on time (found when a 12 s backend reply beat the 10 s timeout) |

#### Bonus A: Customer dashboard aggregation (GET /customers/{customerId}/dashboard)

- MI Scatter-Gather (or Clone + Aggregate) calls Customer and Balance in parallel, with each branch timing out at 5 s.
- Decision: **return partial data**. The response is 200 with a sections object, and a failed section is marked {"status":"UNAVAILABLE","error":{"...}}. If *both* fail, the API returns 503.
- Reason: a dashboard is read-only and still useful without one panel, so failing the whole page for one slow backend is a worse customer experience.

#### Bonus B: Loan application events (RabbitMQ)

- An MI RabbitMQ inbound endpoint consumes loan.applications. It validates and transforms each message, then calls /loans/eligibility and publishes the result to loan.decisions.
- Malformed or unprocessable messages are rejected without requeue and go to loan.applications.dlq through a dead-letter exchange. Transient backend errors are retried 3 times with backoff before going to the DLQ.
- How this differs from the sync APIs: there is no caller waiting, so errors cannot be returned. Instead they must be retried, parked and replayed. Idempotency (an applicationId key) also matters, because a message may be delivered more than once.